04 / GUÍA DE PSEUDOCÓDIGO

# De la transformación geométrica al algoritmo

EX29 · Diseño previo al código

**Para qué sirve**
Antes de programar, identificar matemáticamente qué objeto gira, qué sistema de ejes permanece fijo, qué valores se conocen y qué resultado se espera. Es una guía de organización; no se proporciona la solución Python completa.

## 1. Cuatro preguntas antes de escribir código

1. **¿Qué operación física/matemática se realiza?** ¿Gira el vector o giran los ejes?
2. **¿Cuál es el dato de entrada?** Vector $(v_x, v_y)$, ángulo y unidades.
3. **¿Cuál es la salida?** Componentes rotadas, componentes en otro sistema, norma o comparación.
4. **¿Cómo se comprueba?** Sustitución manual, conservación de norma o inversión de la transformación.

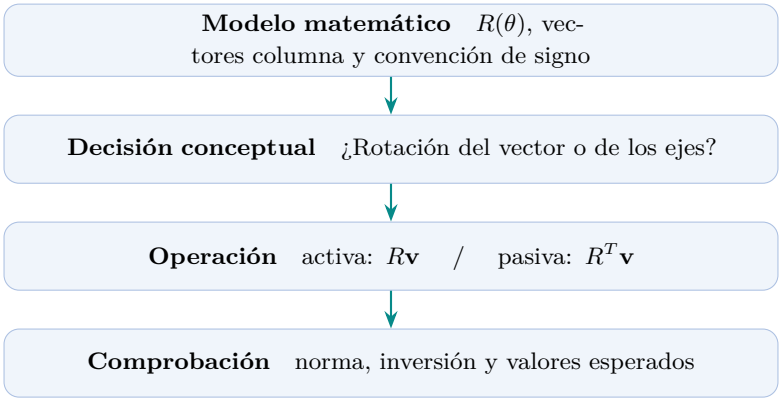
## 2. Pseudocódigo conceptual para una rotación activa

```
INICIO
1.  IDENTIFICAR vector y angulo (+ antihorario).
2.  CONVERTIR angulo a radianes.
3.  CALCULAR seno y coseno.
4.  CONSTRUIR matriz de rotacion.
5.  MULTIPLICAR matriz por vector.
6.  COMPARAR norma original y transformada.
7.  MOSTRAR vector y comprobaciones.
FIN
```

## 3. Pseudocódigo para un cambio de referencia pasivo

```
INICIO
1.  IDENTIFICAR vector en ejes globales.
2.  IDENTIFICAR giro de los ejes (no del vector).
3.  CONSTRUIR la misma matriz R.
4.  USAR la transpuesta R.T para obtener componentes locales.
5.  RECUPERAR coordenadas globales con R.
6.  COMPROBAR componentes y norma.
FIN
```

## 4. Secuencia de razonamiento en una figura




## 5. Cómo dividir los apartados EX29

| Bloque | Qué organizar en el pseudocódigo |
|---|---|
| A0–A2 | Construcción de $R$, multiplicación, comprobación manual e inversión. |
| A3 | Dos matrices sucesivas; comparar con una sola matriz. |
| B | Incluir unidades físicas y verificar magnitud de fuerza. |
| C | Mantener el vector fijo, aplicar la transpuesta y recuperar componentes globales. |

## 6. Consejos de programación

- Utilizar nombres que revelen si una cantidad representa un vector global, local, original o rotado.
- Separar operaciones durante el aprendizaje; no comprimir todo en una única expresión.
- Evitar comparar `float` mediante `==` cuando se esperan aproximaciones trigonométricas.
- Utilizar `shape` durante la depuración si se duda de la dimensión de un array.
- No son necesarias funciones avanzadas, JSON ni tests automáticos en esta clase.